02 — ORM Optimization

Table of Contents

Learning Objectives

By the end of this section you will be able to:

- Identify the N+1 query problem in ORM-generated code and fix it
- Explain why `SELECT *` is harmful in production applications
- Choose between lazy and eager loading in Django, SQLAlchemy, TypeORM, and GORM
- Recognize when an ORM abstraction must be bypassed for raw SQL
- Configure query logging in each major ORM to observe actual SQL
- Analyse EXPLAIN output from ORM-generated queries

The N+1 Problem

What It Is

The N+1 problem occurs when code loads a list of N records and then issues one additional query per record to fetch related data — resulting in N+1 total database round-trips instead of 1 (or 2 at most).

Example: Django ORM Generating N+1

```
# models.py
class Author(models.Model):
    name = models.CharField(max_length=200)

class Book(models.Model):
    title = models.CharField(max_length=300)
    author = models.ForeignKey(Author, on_delete=models.CASCADE)
    published_year = models.IntegerField()
```

```
# BAD: N+1 query
def list_books_bad(request):
```

```
    books = Book.objects.all()      # Query 1: SELECT * FROM books
    result = []
    for book in books:
        result.append({
            'title': book.title,
            'author': book.author.name  # Query 2..N+1: SELECT * FROM authors WHERE
id = ?
        })
    return JsonResponse({'books': result})
```

SQL generated (for 100 books):

```
-- Query 1
SELECT "books"."id", "books"."title", "books"."author_id", "books"."published_year"
FROM "books";

-- Query 2 (repeated 100 times with different author_id)
SELECT "authors"."id", "authors"."name"
FROM "authors"
WHERE "authors"."id" = 1;

SELECT "authors"."id", "authors"."name"
FROM "authors"
WHERE "authors"."id" = 2;
-- ... 98 more identical queries
```

EXPLAIN Output for a Single N+1 Lookup

```
EXPLAIN (ANALYZE, BUFFERS) SELECT "authors"."id", "authors"."name"
FROM "authors" WHERE "authors"."id" = 42;

-- Output:
Index Scan using authors_pkey on authors  (cost=0.28..8.29 rows=1 width=36)
                                          (actual time=0.018..0.019 rows=1 loops=1)
  Index Cond: (id = 42)
  Buffers: shared hit=2
Planning Time: 0.1 ms
Execution Time: 0.04 ms
```

Each individual query is fast (0.04 ms), but 100 round-trips at network latency of 1 ms = **100 ms of pure overhead**.

Fix: JOIN / Eager Load

```
# GOOD: select_related (JOIN)
def list_books_good(request):
    books = Book.objects.select_related('author').all()
    # SELECT books.*, authors.* FROM books INNER JOIN authors ON ...
    result = []
```

```
    for book in books:
        result.append({
            'title': book.title,
            'author': book.author.name  # No extra query — already loaded
        })
    return JsonResponse({'books': result})
```

Single SQL generated:

```
SELECT
    "books"."id", "books"."title", "books"."author_id", "books"."published_year",
    "authors"."id", "authors"."name"
FROM "books"
INNER JOIN "authors" ON ("books"."author_id" = "authors"."id");
```

Fix: prefetch_related (Separate Query + Python Join)

```
# GOOD for reverse FK and M2M: prefetch_related
class Book(models.Model):
    tags = models.ManyToManyField('Tag')

# BAD — 1 + N queries
books = Book.objects.all()
for book in books:
    tags = book.tags.all()  # N extra queries

# GOOD — 2 queries total
books = Book.objects.prefetch_related('tags').all()
for book in books:
    tags = book.tags.all()  # Already in cache
```

SQLAlchemy N+1 Example and Fix

```
from sqlalchemy.orm import Session, selectinload, joinedload

# BAD: lazy loading (default) — N+1
with Session(engine) as session:
    books = session.query(Book).all()
    for book in books:
        print(book.author.name)  # lazy load fires per book

# GOOD: joinedload — single JOIN query
with Session(engine) as session:
    books = session.query(Book).options(joinedload(Book.author)).all()
    for book in books:
        print(book.author.name)  # already loaded

# GOOD: selectinload — 2 queries (SELECT IN), better for collections
with Session(engine) as session:
```

```
    authors = session.query(Author).options(selectinload(Author.books)).all()
    for author in authors:
        for book in author.books:  # already loaded
            print(book.title)
```

TypeORM N+1 Example and Fix

```
// BAD: lazy loading — N+1
const books = await bookRepository.find();
for (const book of books) {
    const author = await book.author; // lazy load, fires N queries
    console.log(author.name);
}

// GOOD: eager via relations option
const books = await bookRepository.find({
    relations: ['author'],  // LEFT JOIN
});
for (const book of books) {
    console.log(book.author.name);  // already loaded
}

// GOOD: QueryBuilder with explicit JOIN
const books = await bookRepository
    .createQueryBuilder('book')
    .leftJoinAndSelect('book.author', 'author')
    .getMany();
```

GORM N+1 Example and Fix

```
// BAD — N+1
var books []Book
db.Find(&books)
for i := range books {
    db.First(&books[i].Author, books[i].AuthorID) // N extra queries
}

// GOOD — Preload (separate query with IN clause)
var books []Book
db.Preload("Author").Find(&books)

// GOOD — Joins (single JOIN query)
var books []Book
db.Joins("Author").Find(&books)
```

SELECT * Pitfalls

Why SELECT * Is Harmful

```sql
-- BAD
SELECT * FROM orders WHERE customer_id = 123;

-- GOOD
SELECT id, order_date, status, total_amount FROM orders WHERE customer_id = 123;
```

**Problems with SELECT :*

1. **Transfers unnecessary data** — a table with 50 columns sends all 50; you may only need 5
2. **Prevents index-only scans** — an index on `(customer_id, status)` covers `SELECT status` but not `SELECT *`
3. **Breaks applications on schema change** — adding or removing a column silently changes the shape of data your code receives
4. **Serialization overhead** — deserializing 50 JSON/ORM columns is more CPU work than 5
5. **Exposes sensitive columns** — passwords, tokens, PII returned to layers that don't need them

ORM Equivalents of SELECT *

```python
# Django — fetch only needed columns
# BAD
books = Book.objects.all()

# GOOD: values() returns dicts
books = Book.objects.values('id', 'title', 'published_year')

# GOOD: only() defers unused columns
books = Book.objects.only('id', 'title')

# GOOD: defer() excludes heavy columns (e.g., large text)
books = Book.objects.defer('description', 'full_text')
```

```python
# SQLAlchemy — specify columns
# BAD
books = session.query(Book).all()

# GOOD
from sqlalchemy import select
stmt = select(Book.id, Book.title, Book.published_year)
books = session.execute(stmt).all()
```

```javascript
// TypeORM — select specific fields
const books = await bookRepository.find({
    select: ['id', 'title', 'publishedYear'],
});

// QueryBuilder
const books = await bookRepository
    .createQueryBuilder('book')
```

```
    .select(['book.id', 'book.title', 'book.publishedYear'])
    .getMany();
```

```
// GORM — select specific columns
var books []Book
db.Select("id", "title", "published_year").Find(&books)
```

Lazy vs Eager Loading

Definitions

Strategy	When Data Loaded	Queries Issued	Use Case
Lazy	On first access	1 + N (if accessed in loop)	When related data is rarely needed
Eager (JOIN)	Same query as parent	1 (larger result set)	Always-needed FK relations, small result sets
Eager (Subquery/SELECT IN)	Separate query after parent	2	Collections, large result sets where JOIN would duplicate rows

Django

```
# Lazy (default for ForeignKey)
book = Book.objects.get(pk=1)
# author not loaded yet
print(book.author.name)  # fires query NOW

# Eager ForeignKey: select_related (JOIN)
book = Book.objects.select_related('author').get(pk=1)
# author loaded in same query

# Eager reverse FK / M2M: prefetch_related (2 queries)
author = Author.objects.prefetch_related('book_set').get(pk=1)
# books fetched in second query: SELECT * FROM books WHERE author_id IN (1)
```

SQLAlchemy

```
from sqlalchemy.orm import relationship, lazyload, joinedload, selectinload,
subqueryload

class Author(Base):
    __tablename__ = 'authors'
    books = relationship('Book', lazy='select')  # default: lazy
```

```python
class Book(Base):
    __tablename__ = 'books'
    author = relationship('Author', lazy='joined')  # always JOIN

# Override at query time
session.query(Book).options(
    joinedload(Book.author),          # JOIN for single FK
    selectinload(Book.tags),          # SELECT IN for M2M
    subqueryload(Author.reviews),     # correlated subquery
).all()
```

TypeORM

```typescript
@Entity()
class Book {
    @ManyToOne(() => Author, { eager: false, lazy: true })
    author: Promise<Author>;  // lazy: wrapped in Promise

    @ManyToMany(() => Tag, { eager: true })  // always loaded
    tags: Tag[];
}

// Query-level override
const books = await bookRepository.find({
    relations: {
        author: true,   // eager for this query
        tags: true,
    },
});
```

GORM

```go
type Book struct {
    gorm.Model
    AuthorID uint
    Author   Author  // loaded via Preload or Joins
    Tags     []Tag   `gorm:"many2many:book_tags;"`
}

// Preload: separate SELECT ... WHERE id IN (...)
db.Preload("Author").Preload("Tags").Find(&books)

// Joins: SQL JOIN (better for filtering)
db.Joins("Author").Where("authors.active = ?", true).Find(&books)

// Nested Preload
db.Preload("Author.Publisher").Find(&books)
```

When to Drop to Raw SQL

Use raw SQL when:

1. **Complex aggregations** — window functions, CTEs, recursive queries
2. **Performance-critical hot paths** — ORM adds overhead; raw SQL + prepared statements is faster
3. **Bulk operations** — `COPY`, `INSERT ... ON CONFLICT DO UPDATE`
4. **Database-specific features** — `LATERAL JOIN`, `UNNEST`, full-text search with rankings, custom operators
5. **Query plan control** — you need specific index hints or join order

Django Raw SQL

```
# Named parameters with %s (positional) or %(name)s (named)
from django.db import connection

with connection.cursor() as cursor:
    cursor.execute("""
        SELECT a.name, COUNT(b.id) as book_count,
               AVG(b.published_year) as avg_year
        FROM authors a
        LEFT JOIN books b ON b.author_id = a.id
        WHERE a.active = %s
        GROUP BY a.id, a.name
        HAVING COUNT(b.id) >= %s
        ORDER BY book_count DESC
    """, [True, 5])
    rows = cursor.fetchall()
    columns = [col[0] for col in cursor.description]
    return [dict(zip(columns, row)) for row in rows]

# Manager method returning model instances
authors = Author.objects.raw("""
    SELECT * FROM authors WHERE id IN (
        SELECT author_id FROM books WHERE published_year >= %s
    )
""", [2020])
```

SQLAlchemy Raw SQL

```
from sqlalchemy import text

with engine.connect() as conn:
    result = conn.execute(text("""
        SELECT author_id, COUNT(*) as cnt,
               array_agg(title ORDER BY published_year DESC) as recent_titles
        FROM books
        WHERE published_year >= :min_year
        GROUP BY author_id
        ORDER BY cnt DESC
```

```
        LIMIT :limit
    """), {"min_year": 2020, "limit": 10})
    return result.mappings().all()
```

TypeORM Raw SQL

```
const result = await dataSource.query(`
    SELECT
        a.id,
        a.name,
        COUNT(b.id)::int AS book_count,
        MAX(b.published_year) AS latest_year
    FROM authors a
    LEFT JOIN books b ON b.author_id = a.id
    GROUP BY a.id, a.name
    HAVING COUNT(b.id) >= $1
    ORDER BY book_count DESC
`, [minimumBooks]);
```

GORM Raw SQL

```
type AuthorStats struct {
    AuthorID  uint
    Name      string
    BookCount int
    AvgYear   float64
}

var stats []AuthorStats
db.Raw(`
    SELECT a.id AS author_id, a.name,
           COUNT(b.id) AS book_count,
           AVG(b.published_year) AS avg_year
    FROM authors a
    LEFT JOIN books b ON b.author_id = a.id
    GROUP BY a.id, a.name
    ORDER BY book_count DESC
    LIMIT ?
`, 10).Scan(&stats)
```

Query Logging Per ORM

Django

```
# settings.py
LOGGING = {
    'version': 1,
    'handlers': {
```

```
        'console': {'class': 'logging.StreamHandler'},
    },
    'loggers': {
        'django.db.backends': {
            'handlers': ['console'],
            'level': 'DEBUG',  # logs ALL SQL
        },
    },
}

# Programmatic — count queries in a block
from django.db import connection, reset_queries
from django.conf import settings
settings.DEBUG = True
reset_queries()
# ... run ORM code ...
print(len(connection.queries))
for q in connection.queries:
    print(q['sql'], q['time'])
```

SQLAlchemy

```
import logging

# Enable SQL echo on engine creation
engine = create_engine("postgresql://...", echo=True)

# Or: set logging level
logging.basicConfig()
logging.getLogger('sqlalchemy.engine').setLevel(logging.INFO)  # shows SQL
logging.getLogger('sqlalchemy.engine').setLevel(logging.DEBUG) # shows parameters
too

# Programmatic — capture queries
from sqlalchemy import event

queries = []

@event.listens_for(engine, "before_cursor_execute")
def before_cursor_execute(conn, cursor, statement, parameters, context,
executemany):
    queries.append({'sql': statement, 'params': parameters})
```

TypeORM

```
// data-source.ts
const AppDataSource = new DataSource({
    type: 'postgres',
    logging: true,          // logs all queries
```

```
    logging: ['query', 'error'],  // specific log levels
    logger: 'advanced-console',
    maxQueryExecutionTime: 1000,  // log queries slower than 1s
});

// Custom logger
import { Logger } from 'typeorm';
class CustomLogger implements Logger {
    logQuery(query: string, parameters?: any[]) {
        console.log(`[SQL] ${query}`, parameters);
    }
    // ... implement other methods
}
```

GORM

```
import (
    "gorm.io/gorm/logger"
    "time"
    "log"
    "os"
)

newLogger := logger.New(
    log.New(os.Stdout, "\r\n", log.LstdFlags),
    logger.Config{
        SlowThreshold:              200 * time.Millisecond,  // slow query threshold
        LogLevel:                   logger.Info,              // Info = all queries
        IgnoreRecordNotFoundError: true,
        Colorful:                  true,
    },
)

db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{
    Logger: newLogger,
})

// Change log level at runtime
db = db.Session(&gorm.Session{Logger: newLogger.LogMode(logger.Silent)})
```

ORM-Generated Query Analysis

Workflow: ORM → SQL → EXPLAIN → Fix

```
# Step 1: Capture the ORM-generated SQL (Django)
qs = Book.objects.filter(published_year__gte=2020).select_related('author')
print(str(qs.query))  # prints SQL without parameters
# Or:
from django.db import connection
```

```
books = list(qs)
print(connection.queries[-1]['sql'])
```

```
-- Step 2: Run EXPLAIN ANALYZE on the captured SQL
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT
    "books"."id", "books"."title", "books"."published_year",
    "authors"."id" AS "authors__id", "authors"."name"
FROM "books"
INNER JOIN "authors" ON ("books"."author_id" = "authors"."id")
WHERE "books"."published_year" >= 2020;

-- Sample output showing a missing index:
Hash Join  (cost=28.50..195.40 rows=450 width=88)
           (actual time=2.3..18.7 rows=450 loops=1)
  Hash Cond: (books.author_id = authors.id)
  ->  Seq Scan on books  (cost=0.00..155.00 rows=450 width=52)
                         (actual time=0.1..8.2 rows=450 loops=1)
        Filter: (published_year >= 2020)
        Rows Removed by Filter: 9550
  ->  Hash  (cost=18.00..18.00 rows=850 width=36)
            (actual time=1.8..1.8 rows=850 loops=1)
        ->  Seq Scan on authors  (cost=0.00..18.00 rows=850 width=36)
                                 (actual time=0.1..0.9 rows=850 loops=1)

-- Problem identified: Seq Scan on books with Filter (not using index)
-- Fix: add index on published_year
CREATE INDEX CONCURRENTLY idx_books_published_year ON books(published_year);
```

Key EXPLAIN Signals to Watch

Signal	Meaning	Action
`Seq Scan` with large row count	Missing index	Add index
`Rows Removed by Filter: N` large	Index not selective enough	Composite index or partial index
`Hash Join` on large tables	OK, but check hash batches	Add `work_mem` if hash batches > 1
`Nested Loop` with large outer	Can be O(N^2)	Prefer Hash Join via index
`cost=X..Y` very high	Planner estimates off	`ANALYZE` table; check `default_statistics_target`
`loops=N` large	Inner side of nested loop called N times	The N+1 at SQL level

---

Common Mistakes

1. **Accessing lazy-loaded relations inside loops** without enabling eager loading — causes N+1.
2. **Using `objects.all()` in a view without pagination** — loads the entire table into memory.
3. **Relying on `SELECT *` in ORM** (`Model.objects.all()`, `find()`), especially on wide tables with JSONB or text columns.
4. **Forgetting that `only()` defers columns** — accessing a deferred field fires a new query per object.
5. **Calling `.count()` after `.all()`** in Django — this fires a new `SELECT COUNT(*)` query; instead, use `len(queryset)` if you already evaluated it, or keep the queryset un-evaluated until needed.
6. **Mutating ORM objects in a loop then saving one-by-one** — use `bulk_update()` or `UPDATE ... FROM`.
7. **Ignoring `explain()` / query log** — never assuming the ORM does what you think it does.
8. **Using ORM relationships as a substitute for database constraints** — always back FKs with actual `FOREIGN KEY` constraints and indexes.

Best Practices

- Always review query logs in development — **turn on SQL logging from day one**
- Use `select_related` / `joinedload` / `Joins` for FK relations you always access
- Use `prefetch_related` / `selectinload` / `Preload` for collections (M2M, reverse FK)
- Specify column lists explicitly: `only()`, `values()`, `select()`, `Select()`
- For bulk reads, use `values_list()` (Django) or Core `select` (SQLAlchemy) — skip ORM model instantiation overhead
- For bulk writes, use `bulk_create()`, `bulk_update()`, `INSERT ... ON CONFLICT`, `COPY`
- Drop to raw SQL for any query that benefits from CTEs, window functions, or database-specific operators
- Write integration tests that assert on query count (`assertNumQueries` in Django)
- Set `maxQueryExecutionTime` in TypeORM to surface slow queries automatically

Performance Considerations

ORM Overhead Comparison (approximate, 10,000 rows)

Operation	Raw psycopg2 (µs)	SQLAlchemy Core (µs)	SQLAlchemy ORM (µs)	Django ORM (µs)
SELECT 10 cols	2,100	2,400	4,800	5,200
Hydration (row→object)	—	minimal	2,300	2,900
INSERT (1 row)	180	210	380	420
bulk_insert (1000 rows)	8,500	9,000	12,000	11,000

Bulk Create Patterns

```
# Django: bulk_create (single INSERT with multiple VALUES)
Book.objects.bulk_create([
    Book(title=f"Book {i}", author_id=1, published_year=2024)
    for i in range(1000)
], batch_size=500)

# SQLAlchemy: core bulk insert
with engine.connect() as conn:
    conn.execute(
        book_table.insert(),
        [{"title": f"Book {i}", "author_id": 1, "published_year": 2024}
         for i in range(1000)]
    )
    conn.commit()
```

Interview Questions & Answers

Q1: What is the N+1 problem and how do you detect it in production?

A: The N+1 problem is when loading N records triggers N additional queries (one per record) to fetch related data. In production, detect it via: slow query logs showing repeated identical queries with different parameter values, query count metrics from ORM instrumentation (e.g., Django's `assertNumQueries`), APM tools (Datadog, New Relic) showing high query count per request, or PgBouncer's `SHOW STATS` showing abnormally high query-to-request ratios.

Q2: When would you use `joinedload` vs `selectinload` in SQLAlchemy?

A: Use `joinedload` for many-to-one (FK) relationships where you expect one related object per row — it produces a single JOIN query. Use `selectinload` for one-to-many or many-to-many collections — it issues a separate `SELECT ... WHERE id IN (...)` query. `joinedload` on collections can produce a Cartesian product (duplicate rows), inflating data transfer; `selectinload` avoids this at the cost of one extra round-trip.

Q3: Why is `SELECT *` dangerous in a production API?

A: It sends unnecessary data over the network, prevents index-only scans, breaks application code silently when columns are added or reordered, exposes sensitive columns that should not reach the application layer, and increases deserialization CPU overhead in the ORM.

Q4: How do you test for N+1 queries in a Django test suite?

A: Use `django.test.TestCase.assertNumQueries(n)` as a context manager. It counts all SQL queries executed within the block and fails the test if the count does not match `n`. Also useful: `django.test.utils.override_settings(DEBUG=True)` combined with inspecting `django.db.connection.queries`.

Q5: What is the difference between `select_related` and `prefetch_related` in Django?

A: `select_related` performs a SQL JOIN and retrieves all related objects in a single query — suitable for ForeignKey and OneToOne fields. `prefetch_related` performs a separate query per relation and joins

results in Python — suitable for ManyToMany and reverse ForeignKey (one-to-many) relations where a JOIN would produce duplicate rows.

Q6: When should you abandon ORM and write raw SQL?

A: When the query requires: CTEs, window functions (`ROW_NUMBER` , `LAG` , `LEAD`), `LATERAL JOIN` , `UNNEST` with arrays, `ON CONFLICT DO UPDATE` , full-text search with ranking, `COPY` for bulk I/O, or when the ORM-generated plan is provably suboptimal and query hints/raw SQL produces a significantly better plan. Also when the ORM abstraction introduces more bugs than it prevents for a specific complex query.

Q7: How does the ORM's N+1 problem manifest at the database level?

A: At the database level you see many small, fast index scans with identical query structure and sequentially incrementing primary key values. `pg_stat_statements` shows a pattern like `calls=N, mean_time=0.05ms` for a query like `SELECT * FROM authors WHERE id = $1` , where N is large and the mean time is tiny. The total time from N calls overwhelms what a single JOIN would cost.

Q8: What is the `only()` / `defer()` tradeoff in Django?

A: `only(fields)` fetches only the specified fields and defers all others. `defer(fields)` fetches all fields except the specified ones. The tradeoff: accessing a deferred field fires a new `SELECT field FROM table WHERE pk = ?` query per object. Use `only()` when you know exactly which fields you need. Use `defer()` to exclude known heavy columns (e.g., a `description TEXT` field) while still getting everything else.

Exercises with Solutions

Exercise 1: Fix N+1 in TypeORM

Problem: The following code causes N+1 queries. Fix it using a QueryBuilder with proper JOINs.

```
async function getOrdersWithItems(): Promise<any[]> {
    const orders = await orderRepository.find();
    const result = [];
    for (const order of orders) {
        const customer = await order.customer;
        const items = await order.items;
        result.push({ order, customer, items });
    }
    return result;
}
```

Solution:

```
async function getOrdersWithItems(): Promise<any[]> {
    return orderRepository
        .createQueryBuilder('order')
        .leftJoinAndSelect('order.customer', 'customer')
        .leftJoinAndSelect('order.items', 'items')
        .leftJoinAndSelect('items.product', 'product')
        .select([
            'order.id', 'order.createdAt', 'order.totalAmount',
            'customer.id', 'customer.name', 'customer.email',
```

```
            'items.id', 'items.quantity', 'items.unitPrice',
            'product.id', 'product.name',
        ])
        .getMany();
}
```

Exercise 2: Diagnose ORM Query with EXPLAIN

Problem: A Django endpoint is slow. You have captured this SQL. Analyze the EXPLAIN output and propose a fix.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT "orders"."id", "orders"."total", "orders"."customer_id", "orders"."status"
FROM "orders"
WHERE "orders"."customer_id" = 12345 AND "orders"."status" = 'pending';
```

```
Seq Scan on orders  (cost=0.00..52840.00 rows=3 width=32)
                    (actual time=45.2..892.3 rows=3 loops=1)
  Filter: ((customer_id = 12345) AND (status = 'pending'))
  Rows Removed by Filter: 2099997
Buffers: shared read=14085
```

Solution:

```
-- Problem: Sequential scan reading 2M rows to find 3 results
-- Fix: Composite index on (customer_id, status)

CREATE INDEX CONCURRENTLY idx_orders_customer_status
ON orders(customer_id, status)
WHERE status = 'pending';  -- partial index even better if 'pending' is minority

-- After index:
-- Index Scan using idx_orders_customer_status on orders
--   (cost=0.56..12.5 rows=3 width=32)
--   (actual time=0.08..0.12 rows=3 loops=1)
--   Buffers: shared hit=4
```

Cross-References

- `01_connection_pooling_deep.md` — Connection pool acquisition from ORMs
- `03_transactions_in_apps.md` — ORM transaction management patterns
- `05_api_database_patterns.md` — Pagination and filtering with ORMs
- `09_java_spring_postgresql.md` — JPA/Hibernate N+1 solutions with @EntityGraph
- `10_nodejs_postgresql.md` — TypeORM and Prisma optimization
- `11_python_postgresql.md` — Django ORM and SQLAlchemy deep optimization
- `12_go_postgresql.md` — GORM optimization and raw pgx queries